

P2249R0

Mixed comparisons for smart pointers

Published Proposal, 2021-01-24

Issue Tracking:

[Inline In Spec](#)

Author:

[Giuseppe D'Angelo](#)

Audience:

SG18, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to enable mixed comparisons for the Standard Library smart pointer class templates `unique_ptr` and `shared_ptr`, so that one can compare them against raw pointers.

Table of Contents

1	Changelog
2	Tony Tables
3	Motivation and Scope
3.1	Associative containers
4	Impact On The Standard
5	Design Decisions
5.1	Should <code>unique_ptr</code> have the full set of ordering operators (<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>), or just <code><=></code> ?
6	Technical Specifications
6.1	Feature testing macro
6.2	Proposed wording
6.2.1	Synopsis
6.2.2	<code>unique_ptr</code>
6.2.3	<code>shared_ptr</code>
7	Implementation experience
8	Acknowledgements
	References
	Informative References
	Issues Index

§ 1. Changelog

- R0
 - First submission

§ 2. Tony Tables

Before	
<pre> class Manager { // The Manager owns the Objects, therefore it uses smart pointers. // The Manager gives out non-owning raw pointers to clients. // A client gives the raw pointer back to the Manager, to tell it // to act on that Object; the Manager has then to look up the Object // in its storage. std::vector<std::unique_ptr<Object>> objects; public: Object* get_object(~~~) const { return objects[~~~].get(); } void drop_object(Object* input) { // Must use erase_if and a custom comparison (e.g. a lambda). auto isEqual = [input](const std::unique_ptr<Object>& o) { return o.get() == input; }; erase_if(objects, input); } ssize_t index_for_object(Object* input) const { // Same story. // Code like this (predicates, etc.) may get duplicated all over the place // where smart pointers are used in containers/algorithms. Surely, // centralizing it is good practice, but there's always the temptation of // just writing the one-liner lambda and "moving on" rather than refactoring... auto isEqual = [input](const std::unique_ptr<Object>& o) { return o.get() == input; }; auto it = std::ranges::find_if(objects, isEqual); // etc. } }; </pre>	<pre> class Manager { std::vect public: Object* g { retur } void drop { // Ju erase } ssize_t i { // Sa auto // et } }; </pre>
<pre> // Suppose instead that the Manager needs to use an associative container rather than // a sequential container (e.g. mapping some data to each object). // Then, an heterogeneous comparator becomes a necessity -- we can't possibly // look up a unique_ptr using another unique_ptr to the same object, especially // if clients give us non-owning raw pointers to act upon. // Heterogeneous comparator template <class T> struct smart_pointer_comparator { struct is_transparent {}; bool operator()(const std::unique_ptr<T>& lhs, const std::unique_ptr<T>& rhs) const { return lhs < rhs; } bool operator()(const std::unique_ptr<T>& lhs, const T* rhs) const { return std::less()(lhs.get(), rhs); } bool operator()(const T* lhs, const std::unique_ptr<T>& rhs) const { return std::less()(lhs, rhs.get()); } }; // A sorted associative container with some data std::map<std::unique_ptr<Object>, Data, </pre>	<pre> // No need fo // ... just u std::map<std: </pre>

Before	
<pre> smart_pointer_comparator<Object>> objects = ~~~; // Heterogeneous lookup using a raw pointer object* ptr = ~~~; auto it = objects.find(ptr); if (it != objects.end()) { use(it->second); } </pre>	<pre> std::less< </pre>
<pre> // Same, with an unordered associative container. // Heterogeneous hasher; [util.smartptr.hash] guarantees that both the // specializations below return the very same value for the same pointer. template <class T> struct smart_pointer_hasher { struct is_transparent {}; size_t operator()(const std::unique_ptr<T>& ptr) const { // equal by definition to std::hash<T*>(ptr.get()), that is, (*this)(ptr.get()) return std::hash<std::unique_ptr<T>>()(ptr); } size_t operator()(T* ptr) const { return std::hash<T*>()(ptr); } }; // Heterogeneous equality comparator template <class T> struct smart_pointer_equal { struct is_transparent {}; bool operator()(const std::unique_ptr<T>& lhs, const std::unique_ptr<T>& rhs) const { return lhs == rhs; } bool operator()(const std::unique_ptr<T>& lhs, const T* rhs) const { return lhs.get() == rhs; } bool operator()(const T* lhs, const std::unique_ptr<T>& rhs) const { return lhs == rhs.get(); } }; std::unordered_map<std::unique_ptr<Object>, Data, smart_pointer_hasher<Object>, smart_pointer_equal<Object>> objects = ~~~; // Heterogeneous lookup Object* ptr = ~~~; auto it = objects.find(ptr); if (it != objects.end()) { use(it->second); } </pre>	<pre> // [P0919R3] // so a custo template <cla { struct is size_t op // eq retur } size_t op retur } }; // Custom het std::unordere smart_poi std::equa // Heterogene Object* ptr = auto it = obj if (it != obj </pre>

§ 3. Motivation and Scope

Smart pointer classes are universally recognized as the idiomatic way to express ownership of a resource (very incomplete list: [\[Sutter\]](#), [\[Meyers\]](#), [\[R.20\]](#)). On the other hand, raw pointers (and references) are supposed to be used as non-owning types to access a resource.

Both smart pointers and raw pointers, as their name says, share a common semantic: **representing the address of an object**.

This semantic comes with a set of meaningful operations; for instance, asking if two (smart) pointers represent the address of the same object. `operator==` is used to express this intent.

Indeed, with the owning smart pointer class templates available in the Standard Library (`unique_ptr` and `shared_ptr`), one can already use `operator==` between two smart pointer objects (of the same class). However one *cannot* use it

between a smart pointer and a raw pointer, because the Standard Library is lacking that set of overloads; instead, one has to manually extract the raw pointer out of the smart pointer class:

```
std::shared_ptr<object> sptr1, sptr2;
object* rawptr;

// Do both pointers refer to the same object?
if (sptr1 == sptr2) { ~~~ } // WORKS
if (sptr1 == rawptr) { ~~~ } // ERROR, no such operator
if (sptr1.get() == rawptr) { ~~~ } // WORKS; but why the extra syntax?
```

This discussion can be easily generalized to the full set of the six relational operations; these operations have already well-established semantics, and are indeed already defined between smart pointers objects (of the same class) or between raw pointers, but they are not supported in mixed scenarios.

We propose to remove this inconsistency by defining the relational operators between the Standard Library owning smart pointer classes and raw pointers.

Allowing mixed comparisons isn't merely a "semantic fixup"; the situation where one has to compare smart pointers and raw pointers commonly occurs in practice (the typical use case is outlined in the first example in the [§ 2 Tony Tables](#) above, where a "manager" object gives non-owning raw pointers to clients, and the clients pass these raw pointers back to the manager, and now the manager needs to do mixed comparisons).

§ 3.1. Associative containers

Moreover, we believe that allowing mixed comparisons is useful in order to streamline heterogeneous comparison in associative containers for smart pointer classes.

The case of an associative container using a `unique_ptr` as its key type is particularly annoying; one cannot practically ever look up in such a container using another `unique_ptr`, as that would imply having two `unique_ptr` objects owning the same object. Instead, the typical lookup is heterogeneous (by raw pointer); this proposal is one step towards making it more convenient to use, because it enables the usage of the standard `std::less` or `std::equal_to`.

We however are not addressing at all the issue of heterogeneous hashing for smart pointers. While likely very useful in general, heterogeneous hashing can be tackled separately by another proposal that builds on top of this one (for instance, by making the `std::hash` specializations for Standard smart pointers 1) *transparent*, and 2) able to hash the smart pointer's `pointer_type` / `element_type*` as well as the smart pointer object itself. But more research and field experience is certainly needed.)

§ 4. Impact On The Standard

This proposal is a pure library extension. It proposes changes to an existing header, `<memory>`, but it does not require changes to any standard classes or functions and it does not require changes to any of the standard requirement tables. The impact is positive: code that was ill-formed before becomes well-formed.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

[\[P0805R2\]](#) is vaguely related to this proposal. It proposes to add mixed comparisons between containers of the same type (for instance, to be able to compare a `vector<int>` with a `vector<long>`), without resorting to manual calls to algorithms; instead, one can use a comparison operator. A quite verbose call to `std::equal(v1.cbegin(), v1.cend(), v2.cbegin(), v2.cend())` can therefore be replaced by a much simpler `v1 == v2`. In this sense, [\[P0805R2\]](#) matches the spirit of the current proposal, although comparing smart pointers and raw pointer does not require any algorithm, and does not have such a verbose syntax.

§ 5. Design Decisions

§ 5.1. Should `unique_ptr` have the full set of ordering operators (`<`, `<=`, `>`, `>=`), or just `<=>`?

[P1614R2] added support for `operator<=>` across the Standard Library. Notably, it *added* `operator<=>` for `unique_ptr`, leaving the other four ordering operators (`<`, `<=`, `>`, `>=`) untouched. On the other hand, when looking at `shared_ptr`, the same paper *replaced* these four operators with `operator<=>`.

We believe that this was done in order to preserve the semantics for the existing operators, which are defined in terms of customization points (notably, `common_type`; `unique_ptr` can work in terms of a custom "fancy" pointer type). We are not bound by any pre-existing semantics, so we are just proposing `operator<=>` for `unique_ptr`.

ISSUE 1 What does LEWG(I) think about this?

§ 6. Technical Specifications

All the proposed changes are relative to [N4868].

§ 6.1. Feature testing macro

Add to the list in [version.syn]:

```
#define __cpp_lib_mixed_smart_pointer_comparisons YYYYMMML // also in <memory>
```

with the value specified as usual (year and month of adoption).

§ 6.2. Proposed wording

§ 6.2.1. Synopsis

Modify [memory.syn] as shown:

```
template<class T1, class D1, class T2, class D2>
    requires three_way_comparable_with<typename unique_ptr<T1, D1>::pointer,
                                         typename unique_ptr<T2, D2>::pointer>
    compare_three_way_result_t<typename unique_ptr<T1, D1>::pointer,
                              typename unique_ptr<T2, D2>::pointer>
    operator<=>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D, class T2>
    requires equality_comparable_with<typename unique_ptr<T1, D>::pointer, T2>
    bool operator==(const unique_ptr<T1, D>& x, const T2& y);
template<class T1, class D, class T2>
    requires three_way_comparable_with<typename unique_ptr<T1, D>::pointer, T2>
    compare_three_way_result_t<typename unique_ptr<T1, D>::pointer, T2>
    operator<=>(const unique_ptr<T1, D>& x, const T2& y);

[...]
```

```
// [util.smartptr.shared.cmp], shared_ptr comparisons
template<class T, class U>
    bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    strong_ordering operator<=>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

template<class T1, class T2>
    requires equality_comparable_with<typename shared_ptr<T1>::element_type*, T2>
    bool operator==(const shared_ptr<T1>& a, const T2& b);
template<class T1, class T2>
    requires three_way_comparable_with<typename shared_ptr<T1>::element_type*, T2>
    compare_three_way_result_t<typename shared_ptr<T1>::element_type*, T2>
    operator<=>(const shared_ptr<T1>& a, const T2& b);

template<class T>
    bool operator==(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    strong_ordering operator<=>(const shared_ptr<T>& x, nullptr_t) noexcept;
```

§ 6.2.2. unique_ptr

In [unique.ptr.special], add after paragraph 11:

```
template<class T1, class D, class T2>
    requires equality_comparable_with<typename unique_ptr<T1, D>::pointer, T2>
    bool operator==(const unique_ptr<T1, D>& x, const T2& y);
12 Constraints: T2 is not a specialization of unique_ptr.
13 Returns: x.get() == y.
template<class T1, class D, class T2>
    requires three_way_comparable_with<typename unique_ptr<T1, D>::pointer, T2>
    compare_three_way_result_t<typename unique_ptr<T1, D>::pointer, T2>
    operator<=>(const unique_ptr<T1, D>& x, const T2& y);
14 Constraints: T2 is not a specialization of unique_ptr.
15 Returns: compare_three_way()(x.get(), y).
```

Renumber the existing paragraphs 12-18 to 16-22.

§ 6.2.3. `shared_ptr`

In `[util.smartptr.shared.cmp]`, insert after paragraph 1:

```
template<class T1, class T2>
requires equality_comparable_with<typename shared_ptr<T1>::element_type*, T2>
bool operator==(const shared_ptr<T1>& a, const T2& b);

2 Constraints: T2 is not a specialization of shared_ptr.
3 Returns: a.get() == b.
```

Renumber the existing paragraphs 2-4 to 4-6, and insert after the (newly numbered) paragraph 6:

```
template<class T1, class T2>
requires three_way_comparable_with<typename shared_ptr<T1>::element_type*, T2>
compare_three_way_result_t<typename shared_ptr<T1>::element_type*, T2>
bool operator<=>(const shared_ptr<T1>& a, const T2& b);

7 Constraints: T2 is not a specialization of shared_ptr.
8 Returns: compare_three_way()(a.get(), b).
```

Renumber the existing paragraph 5 to 9.

§ 7. Implementation experience

A working prototype of the feature described here, done on top of GCC 10.2, is available in this [GCC branch](#) on GitHub.

§ 8. Acknowledgements

Credits for this idea go to Marc Mutz, who raised the question on the [LEWG reflector](#), receiving a positive feedback.

Thanks to the reviewers of early drafts of this paper on the [std-proposals](#) mailing list.

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[MarcMutzReflector]

Marc Mutz. [unique_ptr<T> @ T* relational operators / comparison](#). URL: <https://lists.isocpp.org/lib-ext/2020/07/15873.php>

[Meyers]

Scott Meyers. [Effective Modern C++, Chapter 4. Smart Pointers](#). URL: <https://www.oreilly.com/library/view/effective-modern-c/9781491908419/ch04.html>

[N4868]

Richard Smith. [Working Draft, Standard for Programming Language C++](#). URL: <http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2020/n4868.pdf>

[P0805R2]

Marshall Clow. [Comparing Containers](#). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0805r2.html>

[P0919R3]

Mateusz Pusz. [Heterogeneous lookup for unordered containers](#). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0919r3.html>

[P1614R2]

Barry Revzin. [The Mothership has Landed](#). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1614r2.html>

[P2249-GCC]

Giuseppe D'Angelo. [P2249 prototype implementation](https://github.com/dangelog/gcc/tree/P2249). URL: <https://github.com/dangelog/gcc/tree/P2249>

[R.20]

Bjarne Stroustrup; Herb Sutter. [C++ Core Guidelines, R.20: Use `unique\\_ptr` or `shared\\_ptr` to represent ownership](https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r20-use-unique_ptr-or-shared_ptr-to-represent-ownership). URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r20-use-unique_ptr-or-shared_ptr-to-represent-ownership

[Std-proposals]

[P2249 discussion on the std-proposals mailing list](https://lists.isocpp.org/std-proposals/2021/01/2308.php). URL: <https://lists.isocpp.org/std-proposals/2021/01/2308.php>

[Sutter]

Herb Sutter. [Elements of Modern C++ Style](https://herbsutter.com/elements-of-modern-c-style/). URL: <https://herbsutter.com/elements-of-modern-c-style/>

§ Issues Index

ISSUE 1 What does LEWG(I) think about this? ↵